

**Распределённая и федеративная
оптимизация**

Даня Меркулов

ФКН ВШЭ

Мотивация: данные везде

Почему распределённая оптимизация?

Проблема централизации:

Почему распределённая оптимизация?

Проблема централизации:

Масштаб: 1ЕВ данных — не влезает на один сервер. Решение: разбить данные по N агентам.

Почему распределённая оптимизация?

Проблема централизации:

Масштаб: 1ЕВ данных — не влезает на один сервер. Решение: разбить данные по N агентам.

Конфиденциальность: медицинские данные, банковские транзакции — нельзя отправлять на сервер (GDPR, HIPAA).

Почему распределённая оптимизация?

Проблема централизации:

Масштаб: 1EB данных — не влезает на один сервер. Решение: разбить данные по N агентам.

Конфиденциальность: медицинские данные, банковские транзакции — нельзя отправлять на сервер (GDPR, HIPAA).

Edge devices: смартфоны, IoT-устройства генерируют данные локально. Google Keyboard: 500M+ устройств.

Почему распределённая оптимизация?

Проблема централизации:

Масштаб: 1EB данных — не влезает на один сервер. Решение: разбить данные по N агентам.

Конфиденциальность: медицинские данные, банковские транзакции — нельзя отправлять на сервер (GDPR, HIPAA).

Edge devices: смартфоны, IoT-устройства генерируют данные локально. Google Keyboard: 500M+ устройств.

Параллелизм: ускорение обучения на нескольких GPU/TPU (data parallelism).

Постановка задачи

Задача распределённой оптимизации:

$$\min_{w \in \mathbb{R}^d} F(w) = \frac{1}{N} \sum_{i=1}^N f_i(w)$$

где $f_i(w) = \mathbb{E}_{(x,y) \sim \mathcal{D}_i}[\ell(w; x, y)]$ — локальная функция потерь агента i .

Постановка задачи

Задача распределённой оптимизации:

$$\min_{w \in \mathbb{R}^d} F(w) = \frac{1}{N} \sum_{i=1}^N f_i(w)$$

где $f_i(w) = \mathbb{E}_{(x,y) \sim \mathcal{D}_i}[\ell(w; x, y)]$ — локальная функция потерь агента i .

Ключевая сложность: данные неоднородные (non-IID): $\mathcal{D}_i \neq \mathcal{D}_j$ для разных агентов.

Постановка задачи

Задача распределённой оптимизации:

$$\min_{w \in \mathbb{R}^d} F(w) = \frac{1}{N} \sum_{i=1}^N f_i(w)$$

где $f_i(w) = \mathbb{E}_{(x,y) \sim \mathcal{D}_i} [\ell(w; x, y)]$ — локальная функция потерь агента i .

Ключевая сложность: данные **неоднородные** (non-IID): $\mathcal{D}_i \neq \mathcal{D}_j$ для разных агентов.

Настройка	Данные	Коммуникация	Конфиденциальность
Data Parallelism (облако)	IID (перемешаны)	Быстрая (GPU interconnect)	Нет
Federated Learning	Non-IID (по устройствам)	Медленная (мобильный интернет)	Да

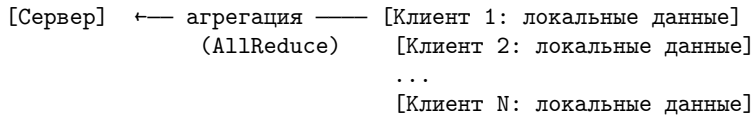
Federated Averaging (FedAvg)

Архитектура Federated Learning

McMahan et al., Google, 2017 — основополагающая работа.

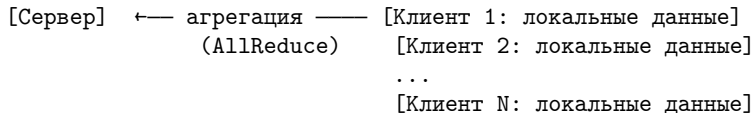
Архитектура Federated Learning

McMahan et al., Google, 2017 — основополагающая работа.



Архитектура Federated Learning

McMahan et al., Google, 2017 — основополагающая работа.



Раунд обучения: 1. Сервер рассылает текущую модель w_t выбранным клиентам 2. Каждый клиент: локальное обучение из w_t за τ шагов SGD 3. Клиенты отправляют обновлённые веса $w_{t,\tau}^{(i)}$ 4. Сервер усредняет:

$$w_{t+1} = \frac{1}{N} \sum_{i=1}^N w_{t,\tau}^{(i)}$$

Алгоритм FedAvg

```
for round  $t = 1, \dots, T$ :  
  Broadcast  $w_t$  to all clients  
  for each client  $i$  in parallel:  
     $w^i_{t,0} = w_t$   
    for step  $k = 0, \dots, \tau-1$ :  
       $w^i_{t,k+1} = w^i_{t,k} - \eta_l \nabla f_i(w^i_{t,k}; \xi^i_{t,k})$   
   $w_{t+1} = 1/N \sum_i w^i_{t,\tau}$     # агрегация
```

Алгоритм FedAvg

```
for round  $t = 1, \dots, T$ :  
    Broadcast  $w_t$  to all clients  
    for each client  $i$  in parallel:  
         $w^i_{t,0} = w_t$   
        for step  $k = 0, \dots, \tau-1$ :  
             $w^i_{t,k+1} = w^i_{t,k} - \eta_l \nabla f_i(w^i_{t,k}; \xi^i_{t,k})$   
         $w_{t+1} = 1/N \sum_i w^i_{t,\tau}$  # агрегация
```

Параметры: - η_l — локальный шаг обучения - τ — число локальных шагов - T — число раундов коммуникации

Алгоритм FedAvg

```
for round  $t = 1, \dots, T$ :  
    Broadcast  $w_t$  to all clients  
    for each client  $i$  in parallel:  
         $w_{i,t,0} = w_t$   
        for step  $k = 0, \dots, \tau-1$ :  
             $w_{i,t,k+1} = w_{i,t,k} - \eta_l \nabla f_i(w_{i,t,k}; \xi_{i,t,k})$   
         $w_{t+1} = 1/N \sum_i w_{i,t,\tau}$  # агрегация
```

Параметры: - η_l — локальный шаг обучения - τ — число локальных шагов - T — число раундов коммуникации

Связь с курсом

FedAvg с $\tau = 1$ = consensus ADMM! (Лекция 12). Теория сходимости ADMM напрямую применима к FL.

Проблема Client Drift

Client drift — главная проблема FedAvg при non-IID данных.

Проблема Client Drift

Client drift — главная проблема FedAvg при non-IID данных.

Иллюстрация: агент 1 видит только кошек (f_1 имеет минимум w_1^*), агент 2 — только собак (f_2 имеет минимум w_2^*). Они тянут модель в разные стороны!

Проблема Client Drift

Client drift — главная проблема FedAvg при non-IID данных.

Иллюстрация: агент 1 видит только кошек (f_1 имеет минимум w_1^*), агент 2 — только собак (f_2 имеет минимум w_2^*). Они тянут модель в разные стороны!

Математически: $\nabla f_i(w) \neq \nabla F(w)$ при non-IID. После τ шагов клиент i уходит в $w_{t,\tau}^{(i)}$ — вдали от оптимума F .

Проблема Client Drift

Client drift — главная проблема FedAvg при non-IID данных.

Иллюстрация: агент 1 видит только кошек (f_1 имеет минимум w_1^*), агент 2 — только собак (f_2 имеет минимум w_2^*). Они тянут модель в разные стороны!

Математически: $\nabla f_i(w) \neq \nabla F(w)$ при non-IID. После τ шагов клиент i уходит в $w_{t,\tau}^{(i)}$ — вдали от оптимума F .

Теорема (сходимость FedAvg, non-IID): постоянное смещение порядка $O(\tau \eta_l^2 \sigma_g^2)$, где σ_g^2 — дисперсия локальных градиентов. При $\tau \rightarrow \infty$ ошибка растёт!

Проблема Client Drift

Client drift — главная проблема FedAvg при non-IID данных.

Иллюстрация: агент 1 видит только кошек (f_1 имеет минимум w_1^*), агент 2 — только собак (f_2 имеет минимум w_2^*). Они тянут модель в разные стороны!

Математически: $\nabla f_i(w) \neq \nabla F(w)$ при non-IID. После τ шагов клиент i уходит в $w_{t,\tau}^{(i)}$ — вдали от оптимума F .

Теорема (сходимость FedAvg, non-IID): постоянное смещение порядка $O(\tau \eta_l^2 \sigma_g^2)$, где σ_g^2 — дисперсия локальных градиентов. При $\tau \rightarrow \infty$ ошибка растёт!

Вывод: FedAvg при non-IID **не сходится к оптимуму** F^* — есть константное смещение.

SCAFFOLD: коррекция дрейфа

Идея SCAFFOLD

SCAFFOLD (Stochastic Controlled Averaging, Karimireddy et al., 2020).

Идея SCAFFOLD

SCAFFOLD (Stochastic Controlled Averaging, Karimireddy et al., 2020).

Идея: добавить **управляющую переменную** c_i для коррекции дрейфа каждого клиента.

Идея SCAFFOLD

SCAFFOLD (Stochastic Controlled Averaging, Karimireddy et al., 2020).

Идея: добавить **управляющую переменную** c_i для коррекции дрейфа каждого клиента.

Переменные: - c — глобальная управляющая переменная (сервер) - c_i — локальная управляющая переменная (клиент i)

Идея SCAFFOLD

SCAFFOLD (Stochastic Controlled Averaging, Karimireddy et al., 2020).

Идея: добавить **управляющую переменную** c_i для коррекции дрейфа каждого клиента.

Переменные: - c — глобальная управляющая переменная (сервер) - c_i — локальная управляющая переменная (клиент i)

Скорректированный локальный шаг:

$$g_i = \nabla f_i(w_{t,k}^{(i)}; \xi) - c_i + c$$

Вычитаем $c_i \approx \nabla f_i - \nabla F \rightarrow$ компенсируем дрейф.

Алгоритм SCAFFOLD

```
for round  $t = 1, \dots, T$ :  
  Broadcast  $w_t, c$  to all clients  
  for each client  $i$  in parallel:  
     $w_{i,\{t,0\}} = w_t$   
    for  $k = 0, \dots, \tau-1$ :  
       $g = \nabla f_i(w_{i,\{t,k\}}; \xi) - c_i + c$       # коррекция  
       $w_{i,\{t,k+1\}} = w_{i,\{t,k\}} - \eta_l g$   
     $c_{i\_new} = c_i - c + (w_t - w_{i,\{t,\tau\}}) / (\tau \eta_l)$   
     $\Delta c_i = c_{i\_new} - c_i$   
 $w_{t+1} = w_t + \eta_g \cdot 1/N \sum_i (w_{i,\{t,\tau\}} - w_t)$   
 $c = c + 1/N \sum_i \Delta c_i$ 
```

Алгоритм SCAFFOLD

```
for round  $t = 1, \dots, T$ :  
  Broadcast  $w_t, c$  to all clients  
  for each client  $i$  in parallel:  
     $\hat{w}_{i,\{t,0\}} = w_t$   
    for  $k = 0, \dots, \tau-1$ :  
       $g = \nabla f_i(\hat{w}_{i,\{t,k\}}; \xi) - c_i + c$  # коррекция  
       $\hat{w}_{i,\{t,k+1\}} = \hat{w}_{i,\{t,k\}} - \eta_l g$   
     $c_{i\_new} = c_i - c + (w_t - \hat{w}_{i,\{t,\tau\}}) / (\tau \eta_l)$   
     $\Delta c_i = c_{i\_new} - c_i$   
   $w_{\{t+1\}} = w_t + \eta_g \cdot 1/N \sum_i (\hat{w}_{i,\{t,\tau\}} - w_t)$   
   $c = c + 1/N \sum_i \Delta c_i$ 
```

Теорема: SCAFFOLD сходится без деградации из-за non-IID:

$$\mathbb{E} \left[\frac{1}{T} \sum_t \|\nabla F(w_t)\|^2 \right] \leq O\left(\frac{1}{\sqrt{T\tau N}}\right)$$

Скорость **не зависит** от σ_g^2 (неоднородности данных)!

FedAvg vs SCAFFOLD

Алгоритм	Non-IID	Доп. коммуникация	Сходимость
FedAvg	☒ смещение $O(\sigma_g^2)$	0	$O(1/\sqrt{KN}) + \text{bias}$
SCAFFOLD	☒ нет смещения	$2\times$ (отправка c_i)	$O(1/\sqrt{KN})$

FedAvg vs SCAFFOLD

Алгоритм	Non-IID	Доп. коммуникация	Сходимость
FedAvg	☒ смещение $O(\sigma_g^2)$	0	$O(1/\sqrt{KN}) + \text{bias}$
SCAFFOLD	☒ нет смещения	$2\times$ (отправка c_i)	$O(1/\sqrt{KN})$

Цена SCAFFOLD: вдвое больше коммуникации (передаём c_i вместе с w).

FedAvg vs SCAFFOLD

Алгоритм	Non-IID	Доп. коммуникация	Сходимость
FedAvg	☒ смещение $O(\sigma_g^2)$	0	$O(1/\sqrt{KN}) + \text{bias}$
SCAFFOLD	☒ нет смещения	$2\times$ (отправка c_i)	$O(1/\sqrt{KN})$

Цена SCAFFOLD: вдвое больше коммуникации (передаём c_i вместе с w).

Вывод

SCAFFOLD = FedAvg + control variates. Принцип такой же как в SVRG (Лекция 10): управляющие переменные исправляют смещение стохастического градиента.

Сжатие коммуникации

Зачем сжимать?

Bottleneck FL: передача весов — главное узкое место.

Зачем сжимать?

Bottleneck FL: передача весов — главное узкое место.

Пример: ResNet-50 с $d = 25M$ параметров. - Один раунд: $25M \times 4 \text{ байт} = 100 \text{ MB}$ на клиента - 1000 клиентов: 100 GB за раунд! - LLaMA-7B: 7×10^9 параметров — 28 GB на клиента

Зачем сжимать?

Bottleneck FL: передача весов — главное узкое место.

Пример: ResNet-50 с $d = 25M$ параметров. - Один раунд: $25M \times 4$ байт = 100 MB на клиента - 1000 клиентов: 100 GB за раунд! - LLaMA-7B: 7×10^9 параметров — 28 GB на клиента

Два подхода: 1. **Спарсификация:** передаём только k из d координат 2. **Квантизация:** снижаем точность каждой координаты

Тор-к спарсификация и Error Feedback

Тор-к спарсификация:

$$\mathcal{C}_k(g) = \text{top-}k \text{ компонент } g \text{ по модулю}$$

Передаём $k \ll d$ координат. Сжатие: d/k раз.

Тор-к спарсификация и Error Feedback

Тор-к спарсификация:

$\mathcal{C}_k(g)$ = top- k компонент g по модулю

Передаём $k \ll d$ координат. Сжатие: d/k раз.

Проблема: $\mathcal{C}_k$ — смещённый компрессор: $\mathbb{E}[\mathcal{C}_k(g)] \neq g$.

Top-k спарсификация и Error Feedback

Top-k спарсификация:

$\mathcal{C}_k(g)$ = top- k компонент g по модулю

Передаём $k \ll d$ координат. Сжатие: d/k раз.

Проблема: $\mathcal{C}_k$ — смещённый компрессор: $\mathbb{E}[\mathcal{C}_k(g)] \neq g$.

Error Feedback (EF-SGD) (Seide et al., 2014):

$e_0 = 0$

for $k = 0, 1, \dots$:

$\tilde{g} = \mathcal{C}(\nabla f(w_k) + e_k)$ # сжимаем + накопленная ошибка

$e_{k+1} = (\nabla f(w_k) + e_k) - \tilde{g}$ # новая ошибка

$w_{k+1} = w_k - \eta \tilde{g}$

Top-k спарсификация и Error Feedback

Top-k спарсификация:

$$\mathcal{C}_k(g) = \text{top-}k \text{ компонент } g \text{ по модулю}$$

Передаём $k \ll d$ координат. Сжатие: d/k раз.

Проблема: $\mathcal{C}_k$ — смещённый компрессор: $\mathbb{E}[\mathcal{C}_k(g)] \neq g$.

Error Feedback (EF-SGD) (Seide et al., 2014):

$$e_0 = 0$$

for $k = 0, 1, \dots$:

$$\tilde{g} = \mathcal{C}(\nabla f(w_k) + e_k) \quad \# \text{ сжимаем + накопленная ошибка}$$

$$e_{k+1} = (\nabla f(w_k) + e_k) - \tilde{g} \quad \# \text{ новая ошибка}$$

$$w_{k+1} = w_k - \eta \tilde{g}$$

Теорема (EF-SGD): $\min_{k < K} \|\nabla f(w_k)\|^2 \leq O\left(\frac{L}{\delta K}\right)$, где $\delta = k/d$ — степень сжатия. Замедление в $1/\delta$ раз.

1-bit SGD и QSGD

1-bit SGD (Seide, Microsoft, 2014): каждую координату — 1 бит.

$$[\mathcal{Q}(g)]_j = \frac{\|g\|_1}{d} \cdot \text{sign}(g_j)$$

Сжатие: $32\times$ (float32 $\rightarrow$ 1 bit). С Error Feedback — сходимость сохраняется!

1-bit SGD и QSGD

1-bit SGD (Seide, Microsoft, 2014): каждую координату — 1 бит.

$$[\mathcal{Q}(g)]_j = \frac{\|g\|_1}{d} \cdot \text{sign}(g_j)$$

Сжатие: $32\times$ (float32 $\rightarrow$ 1 bit). С Error Feedback — сходимость сохраняется!

QSGD (Alistarh et al., 2017): s -уровневая квантизация: - $s = 2$: 1-bit SGD - $s = 8$: 4-bit ($8\times$ сжатие) -
Теоретическая ошибка: $O(\sqrt{d}/s)$

1-bit SGD и QSGD

1-bit SGD (Seide, Microsoft, 2014): каждую координату — 1 бит.

$$[Q(g)]_j = \frac{\|g\|_1}{d} \cdot \text{sign}(g_j)$$

Сжатие: $32\times$ (float32 $\rightarrow$ 1 bit). С Error Feedback — сходимость сохраняется!

QSGD (Alistarh et al., 2017): s -уровневая квантизация: - $s = 2$: 1-bit SGD - $s = 8$: 4-bit ($8\times$ сжатие) -
Теоретическая ошибка: $O(\sqrt{d}/s)$

Метод	Сжатие	Накопл. ошибка	Сходимость
FedAvg (без сжатия)	$1\times$	Нет	☒
Top-k + EF	d/k	Да	☒
1-bit SGD + EF	$32\times$	Да	☒
QSGD ($s = 4$)	$8\times$	Нет	☒

Дифференциальная конфиденциальность

Проблема конфиденциальности

Парадокс FL: мы не передаём данные — только обновления весов $\Delta w^{(i)}$.

Проблема конфиденциальности

Парадокс FL: мы не передаём данные — только обновления весов $\Delta w^{(i)}$.

Но: по обновлениям можно **восстановить** обучающие данные!

Deep Leakage from Gradients (Zhu et al., NeurIPS 2019): по $\nabla \mathcal{L}(w; x, y)$ можно восстановить пиксели изображения (x, y) из **одной** итерации!

Проблема конфиденциальности

Парадокс FL: мы не передаём данные — только обновления весов $\Delta w^{(i)}$.

Но: по обновлениям можно **восстановить** обучающие данные!

Deep Leakage from Gradients (Zhu et al., NeurIPS 2019): по $\nabla \mathcal{L}(w; x, y)$ можно восстановить пиксели изображения (x, y) из **одной** итерации!

Вывод

Передача градиентов $\neq$ защита данных. Нужна формальная гарантия конфиденциальности.

Дифференциальная конфиденциальность (DP)

Определение (Dwork et al., 2006): алгоритм $\mathcal{M}$ является (ϵ, δ) -DP, если для любых соседних датасетов D, D' (отличаются одним примером):

$$\Pr[\mathcal{M}(D) \in S] \leq e^\epsilon \Pr[\mathcal{M}(D') \in S] + \delta \quad \forall S$$

Дифференциальная конфиденциальность (DP)

Определение (Dwork et al., 2006): алгоритм $\mathcal{M}$ является (ϵ, δ) -DP, если для любых соседних датасетов D, D' (отличаются одним примером):

$$\Pr[\mathcal{M}(D) \in S] \leq e^\epsilon \Pr[\mathcal{M}(D') \in S] + \delta \quad \forall S$$

Параметры: - ϵ — privacy budget (меньше = лучше конфиденциальность) - δ — вероятность «провала» гарантии ($\delta \ll 1/n$)

Дифференциальная конфиденциальность (DP)

Определение (Dwork et al., 2006): алгоритм $\mathcal{M}$ является (ϵ, δ) -DP, если для любых соседних датасетов D, D' (отличаются одним примером):

$$\Pr[\mathcal{M}(D) \in S] \leq e^\epsilon \Pr[\mathcal{M}(D') \in S] + \delta \quad \forall S$$

Параметры: - ϵ — privacy budget (меньше = лучше конфиденциальность) - δ — вероятность «провала» гарантии ($\delta \ll 1/n$)

Интуиция: «присутствие одного человека в датасете почти не влияет на результат».

DP-SGD

DP-SGD (Abadi et al., Google Brain, NeurIPS 2016):

```
for k = 0, 1, ...:  
    B = sample(data, batch_size)  
    for each (xi, yi) in B:  
        gi = ∇(w; xi, yi)  
        gi = gi / max(1, ||gi||/C)      # gradient clipping  
    g = 1/|B| * (Σ gi + N(0, σ2C2I)) # Gaussian noise  
    w_{k+1} = wk - η g
```

DP-SGD

DP-SGD (Abadi et al., Google Brain, NeurIPS 2016):

```
for k = 0, 1, ...:  
    B = sample(data, batch_size)  
    for each (xi, yi) in B:  
        gi = ∇(w; xi, yi)  
        gi = gi / max(1, ||gi||/C)      # gradient clipping  
    g = 1/|B| * (Σi gi + N(0, σ2C2I)) # Gaussian noise  
    w_{k+1} = wk - η g
```

Два ключевых шага: 1. **Gradient clipping** (порог C): ограничиваем чувствительность каждого примера 2. **Gaussian noise** (уровень σ): скрываем вклад конкретного примера

DP-SGD

DP-SGD (Abadi et al., Google Brain, NeurIPS 2016):

```
for k = 0, 1, ...:  
    B = sample(data, batch_size)  
    for each (xi, yi) in B:  
        gi = ∇(w; xi, yi)  
        gi = gi / max(1, ||gi||/C)      # gradient clipping  
    g = 1/|B| * (∑i gi + N(0, σ2C2I)) # Gaussian noise  
    w_{k+1} = wk - η g
```

Два ключевых шага: 1. **Gradient clipping** (порог C): ограничиваем чувствительность каждого примера 2. **Gaussian noise** (уровень σ): скрываем вклад конкретного примера

Privacy budget (Moments accountant): $\epsilon \approx \frac{\sigma^{-2} K \ln(1/\delta)}{n^2}$ — растёт с числом итераций K .

Privacy-Utility Tradeoff

ϵ	Шум σ	Точность (MNIST)	Интерпретация
∞	0	99.5%	Без DP
10	0.3	98.7%	Слабое DP
3	1.1	97.2%	Умеренное DP
1	4.0	94.5%	Сильное DP

Privacy-Utility Tradeoff

ϵ	Шум σ	Точность (MNIST)	Интерпретация
∞	0	99.5%	Без DP
10	0.3	98.7%	Слабое DP
3	1.1	97.2%	Умеренное DP
1	4.0	94.5%	Сильное DP

Цена конфиденциальности: $\epsilon = 1 \rightarrow$ потеря $\sim 5\%$ точности на MNIST. Для сложных задач потери больше.

Privacy-Utility Tradeoff

ϵ	Шум σ	Точность (MNIST)	Интерпретация
∞	0	99.5%	Без DP
10	0.3	98.7%	Слабое DP
3	1.1	97.2%	Умеренное DP
1	4.0	94.5%	Сильное DP

Цена конфиденциальности: $\epsilon = 1 \rightarrow$ потеря $\sim 5\%$ точности на MNIST. Для сложных задач потери больше.

Современная практика

Google использует DP-FL (DP-FedAvg) для обучения on-device моделей. Apple — для Siri и QuickType. $\epsilon \approx 8$ считается «хорошим» для текущих LLM.

Сводная таблица FL алгоритмов

Алгоритм	Проблема	Non-IID	Сжатие	Конфид.
FedAvg	Коммуникация	☒ drift	☒	☒
SCAFFOLD	Client drift	☒	☒	☒
FedProx	Нестабильность	Частично	☒	☒
EF-SGD	Коммуникация	☒	☒ Top-k	☒
DP-FedAvg	Инверсия данных	☒	☒	☒
SCAFFOLD + DP + EF	Всё	☒	☒	☒

Сводная таблица FL алгоритмов

Алгоритм	Проблема	Non-IID	Сжатие	Конфид.
FedAvg	Коммуникация	☒ drift	☒	☒
SCAFFOLD	Client drift	☒	☒	☒
FedProx	Нестабильность	Частично	☒	☒
EF-SGD	Коммуникация	☒	☒ Top-k	☒
DP-FedAvg	Инверсия данных	☒	☒	☒
SCAFFOLD + DP + EF	Всё	☒	☒	☒

Практика: чаще всего используют FedAvg + gradient compression. SCAFFOLD — когда важна точность. DP — при строгих требованиях (медицина, финансы).

Связь с предыдущими лекциями

Unifying View: всё — это расщепление

Лекция 12 (ADMM): consensus ADMM — это FedAvg с $\tau = 1$!

Unifying View: всё — это расщепление

Лекция 12 (ADMM): consensus ADMM — это FedAvg с $\tau = 1$!

Лекция 10 (SVRG): управляющие переменные SCAFFOLD = control variates SVRG.

Unifying View: всё — это расщепление

Лекция 12 (ADMM): consensus ADMM — это FedAvg с $\tau = 1$!

Лекция 10 (SVRG): управляющие переменные SCAFFOLD = control variates SVRG.

Лекция 15 (Quasi-Newton): FedNova (Li et al., 2021) использует нормализацию локальных шагов — похоже на preconditioned gradient (как L-BFGS).

Unifying View: всё — это расщепление

Лекция 12 (ADMM): consensus ADMM — это FedAvg с $\tau = 1$!

Лекция 10 (SVRG): управляющие переменные SCAFFOLD = control variates SVRG.

Лекция 15 (Quasi-Newton): FedNova (Li et al., 2021) использует нормализацию локальных шагов — похоже на preconditioned gradient (как L-BFGS).

Лекция 16 (PL-условие): при PL-условии FedAvg сходится без drift к глобальному минимуму! Это активное направление исследований 2024-2025.

Unifying View: всё — это расщепление

Лекция 12 (ADMM): consensus ADMM — это FedAvg с $\tau = 1$!

Лекция 10 (SVRG): управляющие переменные SCAFFOLD = control variates SVRG.

Лекция 15 (Quasi-Newton): FedNova (Li et al., 2021) использует нормализацию локальных шагов — похоже на preconditioned gradient (как L-BFGS).

Лекция 16 (PL-условие): при PL-условии FedAvg сходится без drift к глобальному минимуму! Это активное направление исследований 2024-2025.

Оператор расщепления в FL

Каждый клиент = один оператор в пространстве моделей. Усреднение = consensus шаг. FL — это operator splitting в пространстве параметров!

Резюме лекции

Ключевые идеи:

1. **FedAvg**: локальный SGD + агрегация весов. Прост, но страдает от client drift при non-IID.

Резюме лекции

Ключевые идеи:

1. **FedAvg**: локальный SGD + агрегация весов. Прост, но страдает от client drift при non-IID.

Резюме лекции

Ключевые идеи:

1. **FedAvg**: локальный SGD + агрегация весов. Прост, но страдает от client drift при non-IID.
2. **SCAFFOLD**: управляющие переменные компенсируют drift — сходится без bias независимо от неоднородности данных.

Резюме лекции

Ключевые идеи:

1. **FedAvg**: локальный SGD + агрегация весов. Прост, но страдает от client drift при non-IID.
2. **SCAFFOLD**: управляющие переменные компенсируют drift — сходится без bias независимо от неоднородности данных.

Резюме лекции

Ключевые идеи:

1. **FedAvg**: локальный SGD + агрегация весов. Прост, но страдает от client drift при non-IID.
2. **SCAFFOLD**: управляющие переменные компенсируют drift — сходится без bias независимо от неоднородности данных.
3. **Сжатие (Top-k, 1-bit, QSGD)**: уменьшают коммуникацию в d/k раз; Error Feedback исправляет смещение.

Резюме лекции

Ключевые идеи:

1. **FedAvg**: локальный SGD + агрегация весов. Прост, но страдает от client drift при non-IID.
2. **SCAFFOLD**: управляющие переменные компенсируют drift — сходится без bias независимо от неоднородности данных.
3. **Сжатие (Top-k, 1-bit, QSGD)**: уменьшают коммуникацию в d/k раз; Error Feedback исправляет смещение.

Резюме лекции

Ключевые идеи:

1. **FedAvg**: локальный SGD + агрегация весов. Прост, но страдает от client drift при non-IID.
2. **SCAFFOLD**: управляющие переменные компенсируют drift — сходится без bias независимо от неоднородности данных.
3. **Сжатие (Top-k, 1-bit, QSGD)**: уменьшают коммуникацию в d/k раз; Error Feedback исправляет смещение.
4. **DP-SGD**: gradient clipping + Gaussian noise гарантирует (ϵ, δ) -DP. Цена: потеря точности $\sim 5\%$.

Резюме лекции

Ключевые идеи:

1. **FedAvg**: локальный SGD + агрегация весов. Прост, но страдает от client drift при non-IID.
2. **SCAFFOLD**: управляющие переменные компенсируют drift — сходится без bias независимо от неоднородности данных.
3. **Сжатие (Top-k, 1-bit, QSGD)**: уменьшают коммуникацию в d/k раз; Error Feedback исправляет смещение.
4. **DP-SGD**: gradient clipping + Gaussian noise гарантирует (ϵ, δ) -DP. Цена: потеря точности $\sim 5\%$.

Резюме лекции

Ключевые идеи:

1. **FedAvg**: локальный SGD + агрегация весов. Прост, но страдает от client drift при non-IID.
2. **SCAFFOLD**: управляющие переменные компенсируют drift — сходится без bias независимо от неоднородности данных.
3. **Сжатие (Top-k, 1-bit, QSGD)**: уменьшают коммуникацию в d/k раз; Error Feedback исправляет смещение.
4. **DP-SGD**: gradient clipping + Gaussian noise гарантирует (ϵ, δ) -DP. Цена: потеря точности $\sim 5\%$.
5. **Унификация**: FL = operator splitting в пространстве параметров. FedAvg = consensus ADMM, SCAFFOLD = SVRG в FL.